

Assignment 2: Royal Game of Ur

The second assignment is a programming assignment where you have to implement the reinforcement learning algorithms that you have learned in the course. In particular, you will implement and apply the algorithms to the Royal Game of Ur. This is a 4,500 year old two player game, with both elements of randomness and strategy, where the players compete to score all of their pieces before the other player does so. The assignment consists of multiple exercises that focus on programming the game environment and implementing the algorithms. The code should be written in Python. In addition, there are exercises on the modeling of the environment and analysis of the results. Your findings for these exercises should be written in a small report. This assignment can be made in **pairs**. You have to hand in your Python code as either Jupyter Notebook or regular Python files and the report as PDF together in a single zip file on Canvas under Assignment 2. On the assignment page you can also find a Python Notebook file with starting code for the environment and algorithm. The due date for the second assignment is **4 July at 23:59**. You can earn a total of **27 points** for this assignment, which are divided over the exercises.

Rules of the game

The Royal Game of Ur is a two-player racing board game. It is a historical game that was played around 4,500 years ago in Mesopotamia, and as such is one of the oldest board games in the world. The game is played using two sets of seven game pieces, one set for each player. The goal of the game is to move all pieces from the start to the end of the board, following the path as outlined in Figure 1. In their turn, the player rolls a set of 4 special dice, resulting in an outcome from 0 to 4. The player chooses one of their pieces and moves it a number of tiles equal to the outcome of this roll. Each square can only be occupied by one piece, but the pieces of the opponent can be captured by moving to their square, sending them back to the start. A piece can be scored if it reaches the end of the board, after which it is removed from the board. A player wins when he has scored all of his pieces. In the assignment, we consider the case where each player only has **two** pieces in total, and not seven. This reduces the number of states by a lot, and makes applying the reinforcement learning algorithms easier.

In the sections below, we give a more in depth description of the game. Alternatively, you can read the rules on the following website <https://royalur.net/rules>. Note that we use the Finkel ruleset. On this website you can also play the game against a bot to help understand the rules. You can also watch the following video <https://www.youtube.com/watch?v=WZskjLq040I&t=1s> for a description and playthrough of the game.

The game board

The game board consists of 3 columns of squares. Player 1 starts in the left column, while player 2 starts in the right column. All pieces have to enter the board at the bottom of the 4 squares at the top in the respective column of the player. The path for both players uses the middle column of the board. On these 8 squares, the players can capture each others pieces if they move to a square occupied by a piece of the opponent. At the end of the path, the pieces again go to either the left or right column depending on the player. This means there are 6 “safe” squares

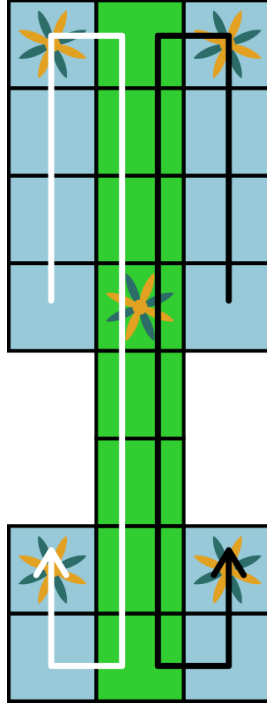


Figure 1: The game board for the Royal Game of Ur

per player and 8 shared “combat” squares. A piece is scored by moving it the exact number of tiles off the board. The board also contains 5 rosette tiles, which grant an extra move if the player moves to them. Furthermore, the rosette in the middle is safe from capture.

Player turn

The players take alternating turns to move their pieces. A turn consists of multiple steps outlined below.

1. Roll the dice: Each player starts their turn by rolling the 4 dice. Each die has an equal probability of turning up 0 or 1. As a result, the score of the dice roll has outcomes 0 through 4 and follows a Binomial distribution with 4 trials and probability $1/2$.
2. Move a piece: The player has to choose to move any of their pieces on the board or to add a new piece to the board. The chosen piece has to move the number of squares equal to the outcome of the dice roll. A piece is only allowed to move to an empty square, except if occupied by the opponent’s piece, in which case the piece can move there to capture it. The player can’t pass a turn, and is thus forced to move a piece if a move is available. If there are no available moves or the roll equals 0, the player doesn’t make a move.
3. Capture a piece: If a player moves its piece to a square occupied by an opponent, then the opponents piece is sent back to the start of the board.
4. Score a piece: To score a piece, a player has to roll the exact number of squares until the end of the board plus one. In this case, the piece can be scored by moving it that number of tiles. Scored pieces are removed for the rest of the game. To give an example of scoring, consider the case where a piece is on the final square

of their path. This square is either one with the rosette at the bottom of the board, as indicated in Figure 1. This piece can then only be scored by rolling a 1.

5. Rosette tiles: If the player moves a piece to one of the 5 rosette tiles, he takes another turn. Furthermore, pieces on the rosette tile in the middle of the board are safe from capture.

End of the game

The objective of the game is for a player to move all seven of their pieces along the course and off the board before their opponent. As already mentioned, we consider the variant in which each player only has two pieces.

Exercises

1. Model the Game of Ur (6 points)

The first exercise is about modeling the Royal Game of Ur. In particular, we want to model the game as an MDP from the perspective of player 1. The goal of player 1 is to maximise his/her chance of winning. We treat player 2 as part of the environment and assume that the policy of player 2 is to always pick randomly between their available actions. This modeling approach is similar to the tic-tac-toe model from the exercise set of week 1. You need to formally define the MDP in terms of the states, actions, and reward function. You don't need to give the transition function, as it is quite complex to formally define, and it is already part of the implementation in the next exercise. You also don't need to give the optimality equation.

- **States:** The state should contain the current information of the game. This consists of the board position for both players, and the current dice roll. There are multiple ways to model the board state, pick a way that you think is best and explain your choice. Furthermore, describe what the terminal states are in this game, and distinguish between the terminal states where player 1 wins and where player 2 wins.
- **Actions:** The action depends on the current state of the game. You have to define the set of possible moves that the player 1 can make given the location of the pieces and their roll. The action itself should consist of the start and end square of the piece to move.
- **Reward function:** Player 1 wishes to maximise his chance of winning the game. As a result, the reward equals 1 when player 1 transitions to a terminal state where he wins, and 0 otherwise. The transition function will have the form $r(s, a, s')$, with current state s , action a , and next state s' , which is slightly different from the usual formulation in the course. It has this form as next state s' determines the reward. Give the exact reward function following this format.
- **Transition function:** You don't need to define the transition function, but we describe its structure below. Transitions always occur between turns of player 1. Generally, the players alternate turns, but due to the existence of rosette tiles, either player can take multiple turns in a row. Thus, a transition looks like this.
 1. We start each epoch on the turn of player 1, and we are given the current state of the MDP, including both board state and dice roll.

2. Player 1 takes an action which updates the board state, and a new dice roll is made.
3. If player 1 moved a piece to a rosette tile, he takes another turn and we go to the next decision epoch, i.e., we go back to step 1. Otherwise, proceed to step 4.
4. It is player 2's turn, who takes a random action from the available actions based on the current state and dice roll. Furthermore, a new dice roll is made.
5. If player 2 moved a piece to a rosette tile, he takes another turn and we go back to step 4. Otherwise, return to step 1.

Thus, the transition contains the effect of the move of player 1, plus the moves of player 2 that happen in between. If either player wins on one of their respective turns, then the MDP transitions to the respective terminal state.

The model formulation should be written down in your report.

2. Implement the game environment (9 points)

In this exercise you will implement the game as a gymnasium environment. As in the tutorials, the environment should have an initialisation function, a reset function, and a step function. The initialisation should take as input the total number of pieces per player, which is two in our case. Furthermore, it may be helpful to define a function that gives the possible moves given the current state, and a function that generates a new dice roll.

You have multiple ways of implementing this environment. You can implement it such that both players can interact with the environment, in which case you need to keep track of the current players turn. Alternatively, you can add the behaviour of player 2 to the gymnasium environment, and let it take turns for player 2 until the next state for player 1 occurs. In either way, think very carefully on how to implement your states and the transitions in the model. Verify that your environment works by making your code play a game, where player 1 now also picks random moves.

3. Implement a reinforcement learning algorithm (7 points)

In this exercise you need to implement and apply a reinforcement algorithm to the Royal Game of Ur. In particular, you have to implement either of the control algorithms n -step SARSA or SARSA(λ). The SARSA(λ) algorithm in the case of no function approximation is not given in the book, but it has a similar structure to the TD(λ) algorithm on slide 28 of lecture 6. The change from TD(λ) to SARSA(λ) is similar to the change from TD(0) to SARSA. In either case, the algorithm needs to optimise the state-action value function $Q(s, a)$. The default step size α and the exploration parameter ϵ are both 0.1, but you can experiment with these values. The discount factor is set to 1, as we are dealing with an episodic task in which the reward is only earned at the end.

4. Analysis and report (5 points)

The final exercise focuses on analysis of the algorithm and reporting on your findings. In particular, we analyse the convergence behaviour of the algorithm. Choose one of the initial state-action pairs (s, a) to keep track of during the running of the algorithm. There are 5 initial states, each with a score of zero for both players and an empty board, but where the states only differ in the outcome of the initial dice roll, which can range from 0 to 4. Each initial state will only have 1 possible action, so there also 5 initial state-action pairs. Perform at least 100,000 episodes of your reinforcement learning algorithm of choice, and save the value of $Q_t(s, a)$ in each iteration t for the state-action pair you have chosen.

In the next part of this exercise, you need to analyse the effect of the parameter values on your algorithm of choice. Depending on your choice of algorithm, you can either modify parameter n or parameter λ . Plot the value function of your initial state for various values of the parameter values. Interpret and elaborate on the results from your plot. In particular, explain how and why the parameter values influence the convergence behaviour.

While not strictly required in your analysis, I also encourage you to experiment with the other parameter values than just n or λ and report on different performance measures than the first-state value. For instance, you could experiment with the step sizes α , or the number of episodes. As for the performance measures, you can also analyse the probability of winning, and how it changes during training.